

Universidad Tecnológica Nacional  
Tecnatura Universitaria en Programación a Distancia

Programación 2

Trabajo Práctico Integrador

Food Store

Sistema de Gestión de Pedidos de Comida

Comisión: 5

Tutor: Renzo Sosa

Alumno: Hernández, María Aldana

11 de julio de 2026

## Tabla de contenido

|                                                                  |    |
|------------------------------------------------------------------|----|
| Introducción                                                     | 3  |
| Objetivos                                                        | 3  |
| Consignas                                                        | 4  |
| Desarrollo                                                       |    |
| Decisiones Técnicas y Arquitectura                               |    |
| Análisis, planificación,<br>diseño e implementación del proyecto | 5  |
| Arquitectura y organización del proyecto                         | 7  |
| Demostración del sistema funcionando                             | 10 |
| Decisiones técnicas y dificultades                               | 12 |
| Conclusiones                                                     | 14 |
| Bibliografía, Webgrafía                                          | 15 |

## Introducción

El presente Trabajo Práctico Integrador consiste en el desarrollo de una aplicación de consola denominada Food Store, destinada a la gestión de productos y pedidos de un negocio de comidas. El sistema fue desarrollado en Java 21, aplicando los principios de la Programación Orientada a Objetos (POO).

La aplicación permite administrar categorías, productos, usuarios y pedidos mediante operaciones CRUD (Crear, Leer, Actualizar y Eliminar) accesibles desde un menú interactivo en consola. No se implementa un sistema de autenticación, por lo que el acceso a las funcionalidades se realiza directamente desde el menú principal.

Toda la información se almacena en memoria utilizando colecciones, sin persistencia en bases de datos. El desarrollo se llevó a cabo siguiendo el backlog de épicas e historias de usuario proporcionado en la consigna, incorporando validaciones de entrada, manejo de excepciones personalizadas y una arquitectura organizada en entidades, servicios y presentación.

## Objetivos

El objetivo de este trabajo es aplicar los conocimientos adquiridos durante la cursada de Programación 2 mediante el desarrollo de una aplicación orientada a objetos que permita evidenciar el uso de buenas prácticas de diseño y programación.

En particular, se busca:

- Modelar correctamente las clases y relaciones definidas en el diagrama UML.
- Aplicar los conceptos de encapsulamiento, herencia y polimorfismo.
- Implementar interfaces para definir comportamientos comunes entre las entidades.
- Utilizar colecciones para administrar la información durante la ejecución del programa.
- Implementar excepciones personalizadas para el manejo de errores y validaciones de negocio.
- Organizar el proyecto mediante una arquitectura por capas, separando entidades, lógica de negocio e interfaz de usuario.
- Desarrollar un sistema de consola estable, con operaciones CRUD completas para las entidades principales del dominio.

## TRABAJO PRÁCTICO INTEGRADOR

### Consignas

El Trabajo Práctico Integrador consiste en desarrollar una aplicación de consola denominada Food Store, destinada a la gestión de un negocio de comidas utilizando el lenguaje Java 21 y los conceptos de Programación Orientada a Objetos.

El sistema debe permitir administrar las entidades Categoría, Producto, Usuario y Pedido mediante operaciones CRUD (Crear, Leer, Actualizar y Eliminar), interactuando con el usuario a través de un menú por consola.

Entre los principales requisitos establecidos por la cátedra se encuentran:

- Implementar el modelo de clases respetando el diagrama UML proporcionado.
- Utilizar una clase base abstracta para compartir atributos comunes entre las entidades.
- Aplicar herencia, encapsulamiento, polimorfismo e interfaces.
- Implementar la interfaz Calculable para el cálculo del total de los pedidos.
- Utilizar colecciones como mecanismo de almacenamiento en memoria, sin emplear bases de datos.
- Implementar operaciones CRUD completas para Categorías, Productos, Usuarios y Pedidos.
- Aplicar baja lógica mediante el atributo eliminado, conservando el historial de información.
- Incorporar validaciones de negocio, como unicidad de datos, control de stock, restricciones sobre cantidades y referencias entre entidades.
- Implementar excepciones personalizadas para el manejo de errores y validaciones.
- Organizar el proyecto siguiendo una arquitectura separada en entidades, servicios e interfaz de usuario.
- Desarrollar una interfaz de consola robusta, con validación de entradas y mensajes claros para el usuario.

Finalmente, el proyecto debía documentarse mediante un informe técnico y una demostración en video, mostrando el funcionamiento completo del sistema y las principales decisiones de diseño adoptadas durante el desarrollo.

### **Análisis, planificación, diseño e implementación del proyecto**

#### **Enfoque de desarrollo**

Para abordar el desarrollo del sistema Food Store, primero se realizó una lectura completa de la consigna, con el objetivo de identificar el alcance del proyecto, las reglas de negocio, las historias de usuario y los criterios de aceptación establecidos.

Debido a que el Trabajo Práctico Integrador constituye una ampliación y continuación del segundo parcial de la materia, se tomó como punto de partida el modelo de clases definido previamente a partir del diagrama UML proporcionado por la cátedra.

El código inicial fue publicado en un repositorio público de GitHub y, antes de incorporar las nuevas funcionalidades, se aplicaron las correcciones y observaciones recibidas durante la evaluación del parcial.

Para organizar el trabajo se creó un tablero de proyecto en GitHub, en el cual se representaron las cuatro épicas principales del sistema: <https://github.com/users/aldimhernandez/projects/4>

- Gestión de categorías.
- Gestión de productos.
- Gestión de usuarios.
- Gestión de pedidos y detalles.

Dentro de cada épica se registraron las historias de usuario correspondientes, permitiendo visualizar el estado de cada tarea y realizar un seguimiento ordenado del avance del proyecto.

El desarrollo se realizó de manera incremental. Inicialmente se diseñó el menú principal y la navegación entre los diferentes submenús, siguiendo la estructura propuesta en la consigna.

Posteriormente, cada historia de usuario se implementó de forma individual, incorporando tanto la interacción correspondiente en el menú como la lógica necesaria en la capa de servicios.

Para cada historia de usuario se creó una rama de trabajo independiente y, una vez finalizada y verificada la funcionalidad, se generó un Pull Request para integrarla al proyecto principal.

Esta forma de trabajo permitió mantener un historial claro de los cambios, revisar cada funcionalidad antes de incorporarla y relacionar las modificaciones del código con las historias de usuario del tablero.

Según las necesidades de cada funcionalidad, también se agregaron validaciones de datos, excepciones personalizadas y métodos específicos dentro de las entidades del dominio. De esta manera, las reglas



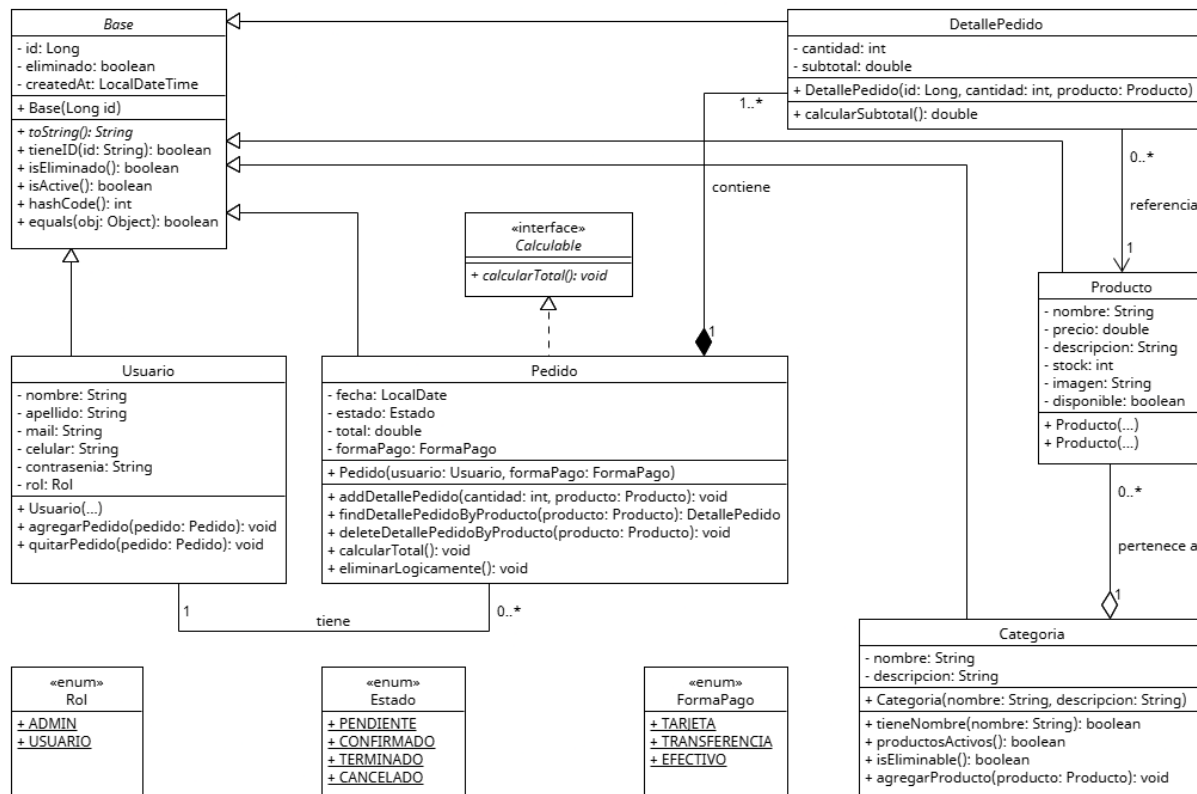
## Arquitectura y organización del proyecto

## Modelado del sistema y entidades

El sistema se basa en un diseño orientado a objetos donde varias clases comparten atributos comunes mediante herencia. Además, se implementan interfaces para definir comportamientos específicos y enumeraciones (enums) para restringir los valores permitidos de determinados atributos, como el rol de los usuarios, los estados de los pedidos y las formas de pago.

La siguiente figura presenta el diagrama de clases (UML) correspondiente al sistema desarrollado. El modelo fue actualizado para reflejar la implementación final del proyecto.

Con el objetivo de mejorar la legibilidad del diagrama, se decidió omitir los métodos getter, setter y algunos métodos heredados de utilidad (como toString(), equals() y hashCode()), representando únicamente los atributos, constructores y métodos de negocio más relevantes de cada entidad.



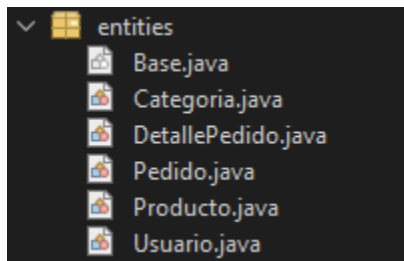
### Estructura del proyecto

El proyecto fue organizado siguiendo una arquitectura por capas, con el objetivo de separar las distintas responsabilidades del sistema y facilitar su mantenimiento, comprensión y extensión.

La aplicación se divide principalmente en tres componentes: el modelo de dominio, la lógica de negocio y la interfaz de usuario por consola. Cada uno de ellos se encuentra distribuido en paquetes específicos.

### Capa de entidades

El paquete entities contiene las clases que representan los objetos principales del sistema:

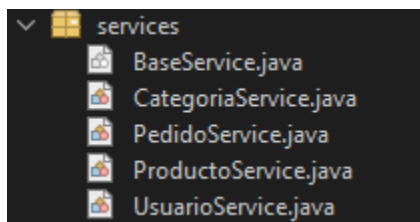


Estas clases definen los atributos, constructores, relaciones y comportamientos propios del dominio. Todas las entidades heredan de la clase abstracta Base, que centraliza los atributos comunes, como el identificador, el estado de eliminación lógica y la fecha de creación.

### Capa de servicios

El paquete services concentra la lógica de negocio y la gestión de las colecciones en memoria.

Cada entidad principal posee un servicio específico:



Además, se implementó una clase genérica BaseService, que reúne operaciones comunes, como la búsqueda de entidades por identificador y la validación de entidades eliminadas.

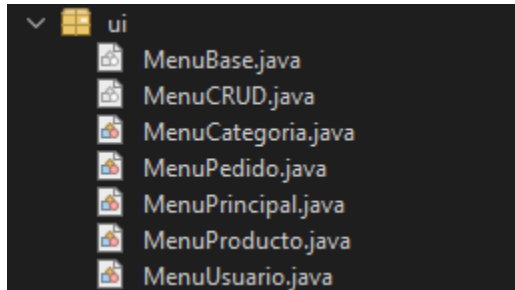
Los servicios actúan como intermediarios entre los menús y las entidades. De esta manera, las reglas de negocio no quedan incorporadas directamente en la interfaz de consola.



## TRABAJO PRÁCTICO INTEGRADOR

### Capa de interfaz de usuario

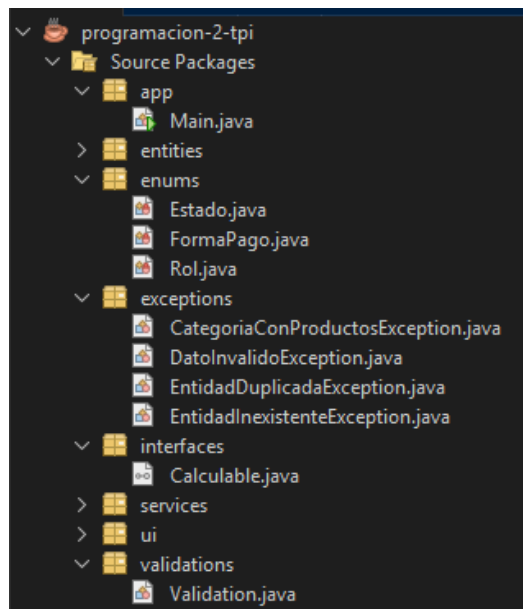
El paquete ui contiene los menús encargados de la interacción con el usuario mediante consola. La navegación comienza en MenuPrincipal, desde donde se accede a los distintos submenús:



También se utilizaron las clases MenuBase y MenuCRUD para centralizar comportamientos comunes, reducir la duplicación de código y mantener una estructura uniforme entre los distintos menús.

### Componentes complementarios

El proyecto incluye otros paquetes destinados a responsabilidades específicas:



app: Contiene la clase Main, punto de entrada de la aplicación.

enums: Define los valores permitidos para el rol, el estado del pedido y la forma de pago.

interfaces: Contiene la interfaz Calculable, implementada por la clase Pedido.

exceptions: Reúne las excepciones personalizadas utilizadas para representar errores del dominio.

validations: Contiene validaciones reutilizables para los datos ingresados.

La clase Main inicializa los servicios y ejecuta el menú principal. Los menús reciben los datos ingresados por el usuario y delegan las operaciones en los servicios. Estos realizan las validaciones necesarias, coordinan las entidades y actualizan las colecciones en memoria.

Esta organización permite mantener separada la interacción por consola de la lógica de negocio y del modelo de dominio, logrando un código más modular, reutilizable y sencillo de mantener.

## TRABAJO PRÁCTICO INTEGRADOR

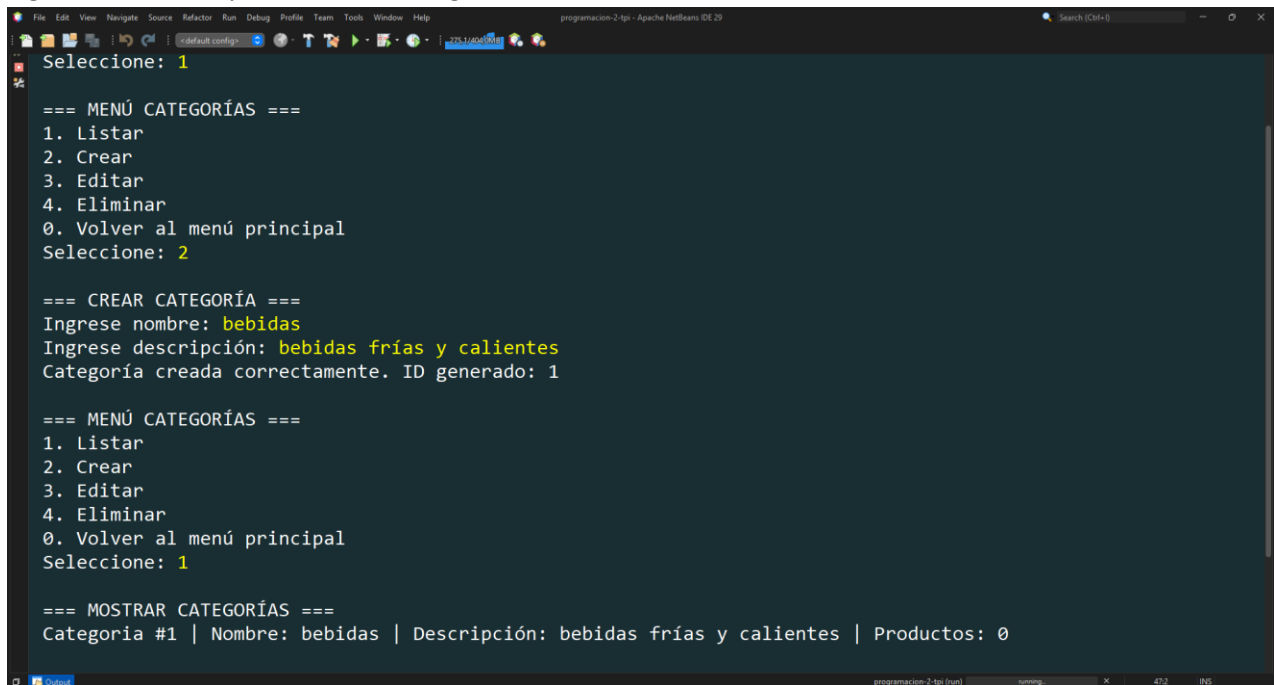
### Demostración del sistema funcionando

A continuación, se presentan dos capturas representativas del funcionamiento del sistema Food Store. Estas evidencias muestran la interacción mediante los menús de consola y la ejecución de algunas de las operaciones principales desarrolladas.

#### Creación y listado de una categoría

La siguiente captura muestra el acceso al menú de categorías, la creación de una nueva categoría y su posterior visualización en el listado general.

Figura 1. Creación y listado de la categoría “Bebidas”.



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
programacion-2-tp1 - Apache NetBeans IDE 20
Seleccione: 1

=== MENÚ CATEGORÍAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione: 2

=== CREAR CATEGORÍA ===
Ingrese nombre: bebidas
Ingrese descripción: bebidas frías y calientes
Categoría creada correctamente. ID generado: 1

=== MENÚ CATEGORÍAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione: 1

=== MOSTRAR CATEGORÍAS ===
Categoría #1 | Nombre: bebidas | Descripción: bebidas frías y calientes | Productos: 0
```

#### Listado de pedidos con usuario y detalles

La siguiente captura presenta el listado de un pedido registrado, incluyendo el usuario asociado, el estado, la forma de pago, la fecha, los detalles de los productos y el total calculado.

## TRABAJO PRÁCTICO INTEGRADOR

Figura 2. Visualización de un pedido con su usuario y detalles asociados.

```
=== MENÚ PEDIDO ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú principal
Seleccione: 1

=== MOSTRAR PEDIDOS ===
> Pedido #1
Usuario asociado:
=====
ID: 1 | USUARIO: aldana hernandez | Mail: aldana@cliente1.com.ar | Rol: USUARIO
=====
Estado: PENDIENTE | FormaPago: TRANSFERENCIA | Fecha: 2026-07-11
-----
- DetallePedido #1: 7up x 20 => Subtotal: $30000,00
TOTAL DEL PEDIDO: $30000,00
-----
```

The image shows a terminal window with a dark background and light-colored text. The text displays a menu for 'MENÚ PEDIDO' with options 1 through 4 and a return to the main menu. Option 1 is selected. This leads to a 'MOSTRAR PEDIDOS' screen showing details for 'Pedido #1'. The details include the user 'aldana hernandez', email 'aldana@cliente1.com.ar', role 'USUARIO', status 'PENDIENTE', payment method 'TRANSFERENCIA', and date '2026-07-11'. A line item shows 'DetallePedido #1: 7up x 20' with a subtotal of '\$30000,00'. The total for the order is also '\$30000,00'. The terminal window has a title bar at the bottom with 'Output' on the left and 'programacion-2.tpr (run)' on the right, along with window control icons and a status bar showing '472' and 'INS'.

Las capturas incluidas tienen como finalidad presentar una evidencia breve del funcionamiento de la aplicación. La demostración completa de las operaciones CRUD, las validaciones, el manejo de errores y los distintos flujos del sistema se encuentra disponible en el video demostrativo incluido al final de este documento.

### **Decisiones técnicas y dificultades**

Durante el desarrollo del proyecto surgieron algunas dificultades relacionadas principalmente con la adaptación al entorno de trabajo y con la aplicación práctica de determinados conceptos de Java.

### **Adaptación al entorno NetBeans y Git**

Una de las primeras dificultades fue la adaptación al IDE NetBeans, debido a que habitualmente utilizo Visual Studio Code y su interfaz integrada para trabajar con Git. En ese entorno ya estaba familiarizada con la revisión de cambios, la creación de commits, la ejecución de operaciones de push y pull, y la administración general del repositorio.

Al comenzar a utilizar NetBeans, algunas de estas tareas resultaron inicialmente menos intuitivas, especialmente la identificación de las modificaciones realizadas en cada archivo antes de escribir mensajes descriptivos para los commits. Esta dificultad se resolvió mediante la práctica progresiva con las herramientas de control de versiones incorporadas en el IDE. A medida que avanzó el proyecto, fue posible integrar NetBeans con el flujo de trabajo utilizado en GitHub, basado en ramas, commits y Pull Requests asociados a cada historia de usuario.

### **Problema de compilación al reorganizar la clase principal**

Durante la reorganización de los paquetes se creó el paquete app y se trasladó allí la clase Main. Luego de este cambio, el proyecto dejó de ejecutarse correctamente, aunque el IDE no mostraba errores claros relacionados con la lógica del código.

Como solución temporal, se revisaron las propiedades del proyecto y se desactivó la opción Compile on Save, ubicada en la configuración de compilación de NetBeans. Esto permitió volver a compilar y ejecutar el sistema, por lo que el desarrollo pudo continuar normalmente.

Posteriormente, después de reiniciar el entorno de trabajo, se volvió a activar esta opción y el proyecto continuó funcionando sin presentar el inconveniente. Por este motivo, se interpretó que el problema se encontraba relacionado con el estado de compilación o la actualización interna del IDE luego de mover la clase principal, y no con una falla en la implementación del sistema.

### **Manejo de excepciones e implementación de búsquedas**

Otra dificultad estuvo relacionada con la aplicación práctica de algunos recursos propios de Java, especialmente el manejo de excepciones mediante estructuras try-catch y excepciones personalizadas.

## TRABAJO PRÁCTICO INTEGRADOR

Si bien los conceptos de Programación Orientada a Objetos ya resultaban familiares, fue necesario adquirir mayor práctica para determinar en qué nivel debía lanzarse una excepción y en qué parte del programa debía capturarse para informar el error sin interrumpir la ejecución del menú.

También presentó cierta dificultad la implementación del método de búsqueda por identificador en la clase BaseService.

Inicialmente, las búsquedas dentro de las colecciones se realizaban mediante ciclos `forEach`. Sin embargo, siguiendo los criterios trabajados durante la cursada, este procedimiento fue reemplazado por el uso de un `Iterator` y una estructura `while`.

La condición del ciclo contempla tanto la existencia de elementos pendientes mediante `hasNext()` como el estado de la búsqueda, permitiendo finalizar la iteración una vez encontrada la entidad correspondiente. Para realizar esta implementación fue necesario revisar la bibliografía y el material audiovisual de la clase.

Una vez comprendido y aplicado el procedimiento en BaseService, la misma solución se reutilizó en otros métodos de búsqueda del sistema. Esto permitió mantener un criterio uniforme para recorrer las colecciones.

### Conclusiones

La realización de este Trabajo Práctico Integrador resultó una experiencia muy enriquecedora, ya que permitió desarrollar mi primera aplicación en Java de una complejidad y alcance mayores a los proyectos realizados anteriormente.

A lo largo del trabajo fue posible integrar y aplicar de manera conjunta numerosos conceptos abordados durante la cursada, entre ellos la herencia, las clases abstractas, las interfaces, las colecciones, las excepciones personalizadas y el manejo de errores mediante estructuras try-catch.

En particular, trabajar con clases abstractas e interfaces representó una experiencia nueva que permitió comprender mejor su utilidad para definir estructuras comunes, reutilizar comportamientos y organizar las responsabilidades dentro del sistema.

La dimensión del proyecto también hizo necesario volver sobre la bibliografía y los materiales de clase para revisar conceptos que, durante su presentación teórica, no habían sido comprendidos completamente. Poder aplicarlos dentro de una situación concreta facilitó su comprensión y permitió reconocer con mayor claridad cuándo y por qué utilizar cada recurso.

Además del resultado técnico, el proyecto permitió adquirir mayor seguridad para analizar requerimientos, organizar el desarrollo mediante historias de usuario y resolver problemas de manera progresiva. Considero que esta experiencia me brindó una base sólida para afrontar nuevos desafíos relacionados con el desarrollo de software.

En conclusión, el trabajo no solo permitió cumplir con los objetivos técnicos propuestos, sino también consolidar los conocimientos adquiridos durante la materia y comprender su aplicación dentro de un sistema completo. Esta experiencia resultó especialmente valiosa y motivadora, ya que demostró que cuento con las bases necesarias para continuar aprendiendo y asumir proyectos de mayor complejidad.

## TRABAJO PRÁCTICO INTEGRADOR

### **Bibliografía, Webgrafía**

Para el desarrollo del proyecto se utilizaron como material de consulta y apoyo los siguientes recursos:

### **Referencias**

Universidad Tecnológica Nacional. Material de estudio de la asignatura Programación 2. Tecnicatura Universitaria en Programación a Distancia, 2026.

Cimino, Charly. Coordinador de la carrera.

Videos y clases de apoyo sobre Programación Orientada a Objetos y desarrollo en Java. Material audiovisual consultado durante la cursada: <https://www.youtube.com/@CharlyCimino>

### **Enlaces del proyecto**

#### **Repositorio de GitHub**

Código fuente completo del sistema Food Store:

Enlace:

<https://github.com/aldimhernandez/hernandez-aldana-programacion-2-tpi-java-poo-food-store>

#### **Tablero de planificación**

Utilizado para organizar las épicas, historias de usuario y estados de avance del proyecto:

Enlace: <https://github.com/users/aldimhernandez/projects/4>

#### **Video demostrativo**

Video de presentación y demostración del funcionamiento completo del sistema, incluyendo la navegación por los menús, las operaciones CRUD, las validaciones y el manejo de excepciones.

Enlace: <https://youtu.be/40kbTrFFPh4>